

LABORATORIO DI INGEGNERIA DEI SISTEMI SOFTWARE

Introduction

Questo documento descrive il lavoro svolto nello **Sprint 1** per lo sviluppo del sistema **cargoservice**.

Goal dello Sprint 1

L'obiettivo (**Goal**) dello Sprint 1 è la realizzazione dell'architettura logica di controllo fondamentale del servizio (l'attore orchestratore `cargoservice`) e il pilotaggio del robot virtuale `RobotSmart26` tramite un driver di traduzione (l'attore `cargorobot`) per superare l'abstraction gap cartesiano. Il corretto funzionamento della logica (Happy Path del deposito container e rifiuto a stiva piena) deve essere validato tramite test di integrazione automatici in JUnit.

Requirements

Il testo dei requisiti finali del tema è consultabile direttamente nel file di specifica fornito dal committente: [Tema2026.html](#) ([../Tema2026.html](#))

Requirement analysis | Vocabolario

Nello Sprint 0 sono state chiarite le entità e mappate sui concetti formali di QActor, Contesto e Messaggi. La struttura a 3 contesti definita nello Sprint 0 rimane la nostra architettura logica di riferimento. In questo sprint implementeremo i primi due attori reali (`cargoservice` e `cargorobot`) mantenendo mockati gli input distribuiti (sonar e led) e l'interfaccia utente (WebGUI).

Problem analysis

Stato della Stiva (slots1-4 e slot5)

L'orchestratore deve memorizzare se ciascuno dei quattro slot di stoccaggio (`slots1-4`) è libero o occupato, oltre ad attendere la marcatura nello `slot5`. Poiché questo stato è puramente informativo e interno alla logica di business di `cargoservice`, viene modellato come un insieme di variabili locali booleane all'interno dello stato dell'attore QActor `cargoservice`, evitando l'introduzione di un componente o attore separato per la stiva:

```
// Definizione variabili locali in cargoservice.qak
var Slot1Occupied = false
var Slot2Occupied = false
var Slot3Occupied = false
var Slot4Occupied = false
var SystemOutOfService = false
var IOPortOccupied = false
var ReservedSlot = ""
```

Risoluzione dell'Abstraction Gap per RobotSmart26

Il robot virtuale **RobotSmart26** (che risiede nel contesto esterno `ctxrobotsmart` alla porta `8020`) risponde a comandi cartesiani e di basso livello. Le nostre tappe logiche sono definite come costanti nominali: `ioport`, `slot5`, `slot1`, `slot2`, `slot3`, `slot4` e `home`. Esiste un evidente **abstraction gap** tra queste destinazioni semantiche e i comandi cartesiani richiesti dal robot.

Questo divario viene risolto all'interno del QActor **cargorobot**, che funge da driver di traduzione. Il `cargorobot` mantiene internamente una mappatura statica per tradurre le posizioni logiche in coordinate reali (X, Y) all'interno della griglia della stanza virtuale:

- `home` → `(0,0)`
- `ioport` → `(4,0)`
- `slot5` → `(4,3)`
- `slot1` → `(1,1)`
- `slot2` → `(1,4)`
- `slot3` → `(3,2)`
- `slot4` → `(3,4)`

Quando riceve la richiesta `move_robot(DEST)` da `cargoservice`, il `cargorobot` traduce `DEST` nelle coordinate corrispondenti e invia al `RobotSmart26` la richiesta formale di movimento:

```
// Traduzione e richiesta cartesiana in cargoservice.qak
State doMove {
  onMsg(move_robot : move_robot(DEST)) {
    [#
      CurrentDest = payloadArg(0)
      when (CurrentDest) {
        "home" -> { TargetX = 0; TargetY = 0 }
        "ioport" -> { TargetX = 4; TargetY = 0 }
        "slot5" -> { TargetX = 4; TargetY = 3 }
        "slot1" -> { TargetX = 1; TargetY = 1 }
        "slot2" -> { TargetX = 1; TargetY = 4 }
        "slot3" -> { TargetX = 3; TargetY = 2 }
        "slot4" -> { TargetX = 3; TargetY = 4 }
        else -> { TargetX = 0; TargetY = 0 }
      }
    #]
    println("$name | translating destination $CurrentDest to ($TargetX, $TargetY)") color green
    request robotsmart -m moverobot : moverobot($TargetX, $TargetY, $StepTime)
  }
}
```

Il `cargorobot` attende la risposta del robot (`moverobotdone` o `moverobotfailed`) e risponde a sua volta a `cargoservice` con il risultato della movimentazione.

Analisi dell'interazione distributiva

Nel metamodello QAK (e nell'Actor Model in generale), **tutte le comunicazioni fisiche a scambio di messaggi avvengono in modo asincrono** (non-blocking).

- I messaggi di tipo `Dispatch` sono nativamente asincroni e non bloccanti (es. l'avvio della marcatura tramite `start_marking` o l'aggiornamento del Led/Display).
- La comunicazione sincrona di tipo `Request-Reply` (es. la richiesta di movimento inviata da `cargoservice` a `cargorobot`) viene realizzata fisicamente tramite due comunicazioni asincrone separate (una Request di andata e un Reply di ritorno). L'attore mittente simula il comportamento sincrono bloccando la propria transizione di stato fino alla ricezione della risposta, rilasciando al contempo il thread fisico di esecuzione.

I contratti formali di messaggistica asincrona e di request-reply definiti per il coordinamento sono:

```
// Messaggi dichiarati in cargoservice.qak
Request move_robot : move_robot(DEST)
Reply move_done : move_done(RESULT) for move_robot

Request moverobot : moverobot(TARGETX, TARGETY, STEPTIME)
Reply moverobotdone : moverobotdone(ARG) for moverobot
Reply moverobotfailed: moverobotfailed(PLANDONE, PLANTODO) for moverobot
```

Test plans

Per validare la correttezza del comportamento del sistema nello Sprint 1, dobbiamo definire dei test automatizzati che avviino il sistema e simulino l'interazione del cliente. I test saranno scritti in Kotlin/Java tramite **JUnit 4**.

Test Funzionale 1: Carico Rifiutato (Stiva Piena)

Stato: Il `cargoservice` registra che gli slot da 1 a 4 sono occupati.

Azione: Invio richiesta `load` tramite CoAP.

Risultato Atteso: Il sistema risponde immediatamente con il messaggio `retrylater` o `reject` senza attivare la movimentazione robotica.

Test Funzionale 2: Accettazione Carico (Happy Path)

Stato: Stiva parzialmente vuota e sistema pienamente funzionante.

Azione: Si invia una richiesta `load` a stiva vuota. Si verifica che la risposta indichi lo slot assegnato (es. `slot1`). Si simula il rilevamento del container inviando il messaggio del sonar. Si osserva (via CoAP) che il robot esegua la sequenza completa e ritorni in `home`, occupando lo `slot1`.

Project

La progettazione e l'implementazione reale del codice dello Sprint 1 sono disponibili nei seguenti file:

- Modello eseguibile reale: `sprint1/src/cargoservice.qak`
(<https://github.com/riccardocar99/cargoservice-/blob/main/sprint1/src/cargoservice.qak>)
- Codice del test automatico: `sprint1/src-test/TestSprint1.kt`
(<https://github.com/riccardocar99/cargoservice-/blob/main/sprint1/src-test/TestSprint1.kt>)

Testing

I test automatici JUnit sono stati eseguiti con successo. L'output del comando `./gradlew test` conferma che tutti i test JUnit definiti (Happy Path e rifiuto stiva piena) passano regolarmente, validando la logica dello Sprint 1.

```
> Task :test
test.kotlin.TestSprint1 > testHappyPath PASSED
```

```
BUILD SUCCESSFUL in 10s
```

Deployment

Per eseguire il sistema nello Sprint 1:

1. Avviare il container Docker del robot virtuale **RobotSmart26** sulla porta `8020`.
2. Posizionarsi nella directory `sprint1` ed eseguire il comando `./gradlew run` per avviare il servizio principale.

Maintenance

(N/A per Sprint 1)

Luigi



Riccardo



Simone



GIT repo: <https://github.com/riccardocar99/cargoservice->
(<https://github.com/riccardocar99/cargoservice->)